

Figure 1: A continuous integration architecture

computer to execute multiple slave agents.

The master-slave set-up is very helpful when you need to execute the job on different types of operating systems. Consider a situation in which the application is expected to run on Linux, Solaris and Windows. To address this need, you can install Hudson on any machine, with Linux for instance, and add two slaves: with Solaris and Windows. The user can add three different jobs—one running on the master and others running on slave machines.

When slaves are registered to a master, the master starts distributing loads to slaves. The delegation depends on the specific job. You can either configure a job to execute on the master, or tie it to a specific slave. On the other hand, you can configure jobs to freely roam between slaves—i.e., the job will execute on the slave that's free of any load. For the user, this set-up is transparent—you can view results for all jobs using the master, irrespective of the slave that executed the jobs.

You can launch a slave from a master using any of the following methods (Figure 4):

1. Launch slave agents on UNIX/Linux machines using SSH.
2. Launch slave agents via JNLP.
3. Launch slave agents by executing a command on the master.
4. Let Hudson control Windows slaves as a Windows service.

Launching slaves on UNIX via SSH

Hudson has a built-in SSH client implementation that can be used to talk to remote `sshd` daemons to start a slave agent. This is the most convenient and preferred method for UNIX slaves, which normally have SSH servers running out-of-the-box.

Click **Manage Hudson** → **Manage Nodes** → **New Node**. In this set-up, the connection information is supplied, including the SSH host name, user



Figure 2: The Hudson home page after configuring multiple jobs

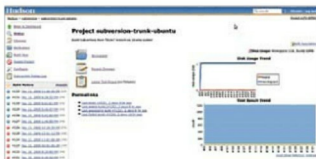


Figure 3: The job dashboard



Figure 4: List of launch methods for slaves



Figure 5: Slave configuration screen

name and authentication credentials. You'll need to authenticate using the password of the remote (slave) UNIX system, or SSH keys. If it is configured to use SSH keys, the SSH public key should be copied to the `~/ssh/authorized_keys` file. Hudson will do the rest of the work by itself, including copying the binary needed for a slave agent, and starting/stopping slaves.

In short, when you launch a slave using this method, a new Java process is started in the slave, automatically. Isn't it the most convenient set-up on UNIX systems?



Tips: Depending on the project and the availability of hardware resources, someone's desktop can be used as one of the slaves, without affecting his or her day-to-day activities. This will avoid the requirement for dedicated slaves.

Launching the slave agent via Java Web Start

Another way of launching a slave is to start a slave agent